

```

79.
80.     # Create offspring through crossover and mutation
81.     while len(next_generation) < population_size:
82.         # Select two parents
83.         parent1, parent2 = random.sample(top_performers, 2)
84.
85.         # Crossover (averaging parameters)
86.         short_ma = (parent1[0] + parent2[0]) // 2
87.         long_ma = (parent1[1] + parent2[1]) // 2
88.
89.         # Mutation (small random adjustments)
90.         if random.random() < 0.3: # 30% mutation probability
91.             short_ma += random.randint(-5, 5)
92.             long_ma += random.randint(-10, 10)
93.
94.         # Ensure parameters are within bounds
95.         short_ma = max(param_bounds['short_ma'][0], min(short_ma,
param_bounds['short_ma'][1]))
96.         long_ma = max(param_bounds['long_ma'][0], min(long_ma, param_bounds['long_ma']
[1]))
97.
98.         # Ensure short_ma is less than long_ma
99.         if short_ma >= long_ma:
100.             long_ma = short_ma + random.randint(10, 30)
101.             long_ma = min(long_ma, param_bounds['long_ma'][1])
102.
103.         next_generation.append((short_ma, long_ma))
104.
105.     # Set next generation as current population
106.     population = next_generation
107.
108.     # Evaluate final population
109.     final_results = []
110.     for params in population:
111.         short_ma, long_ma = params
112.
113.         # Calculate indicators
114.         data['SMA_Short'] = data['Adj Close'].rolling(window=short_ma).mean()
115.         data['SMA_Long'] = data['Adj Close'].rolling(window=long_ma).mean()

```